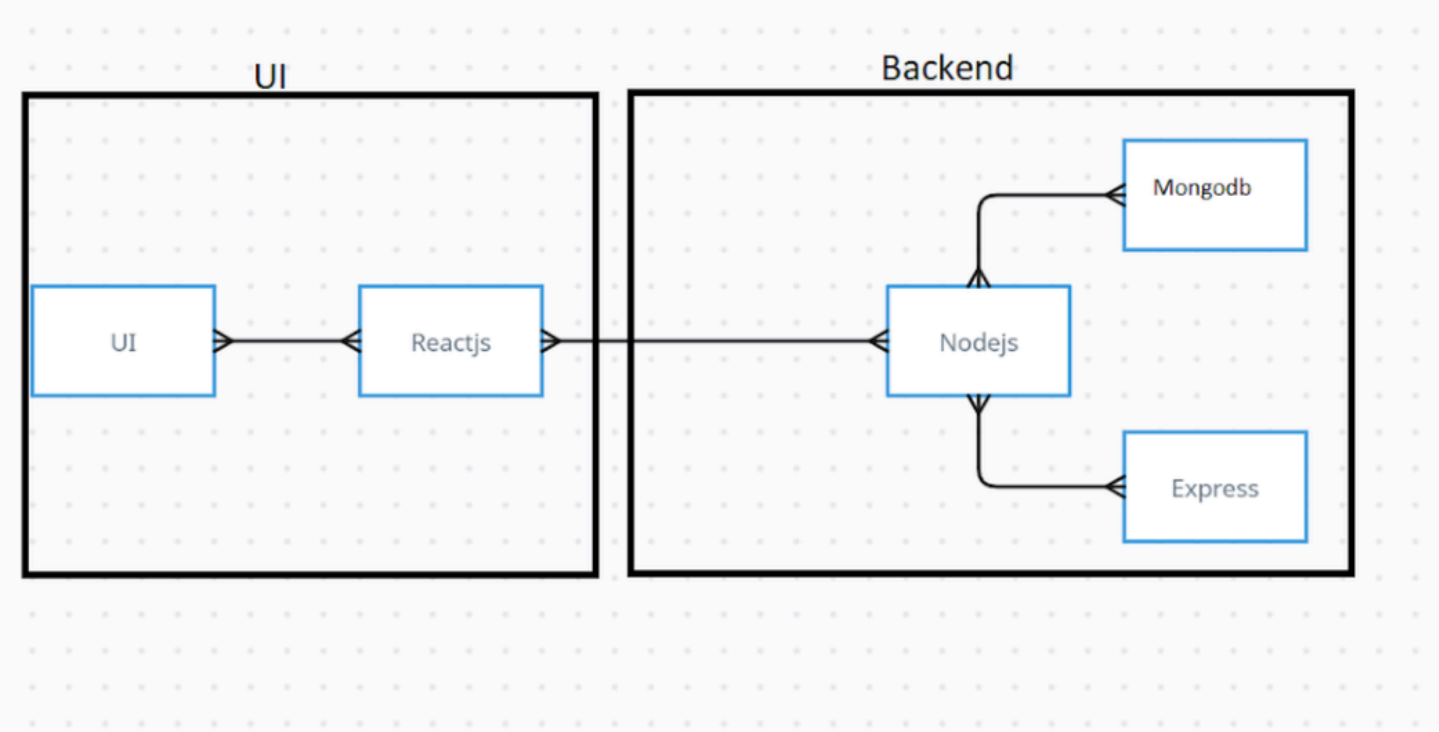
Technical Architecture

The technical architecture of the **House Rent Application** follows a **client–server model**, where the **frontend acts as the client** and the **backend functions as the server**. This architecture enables efficient communication between the user interface and the server-side logic, ensuring a seamless experience for all users.



Frontend Architecture

The frontend of the application is responsible for handling the **user interface, presentation layer, and client-side interactions**. It is built using **React.js**, which allows the development of dynamic and responsive user interfaces through reusable components.

The UI is designed to support different types of users, including **administrators and regular users**, ensuring accessibility and usability across the platform.

Backend Architecture

The backend of the application is developed using **Node.js**. It processes incoming requests from the frontend, performs necessary operations, and sends appropriate responses back to the client.

The backend exposes **RESTful APIs** that manage key functionalities of the system, such as:

- User registration and authentication
- Adding rental properties or rooms

- Viewing available rental listings
- Booking rental rooms
- Managing user profiles and property details

Database Management

For data storage and retrieval, the application uses **MongoDB**, a NoSQL database designed for scalability and flexibility. MongoDB stores various types of application data, including:

- User profiles
- Property/room listings
- Booking information
- Administrative data

MongoDB enables efficient data management through document-based storage, allowing the system to quickly access and update information when required.

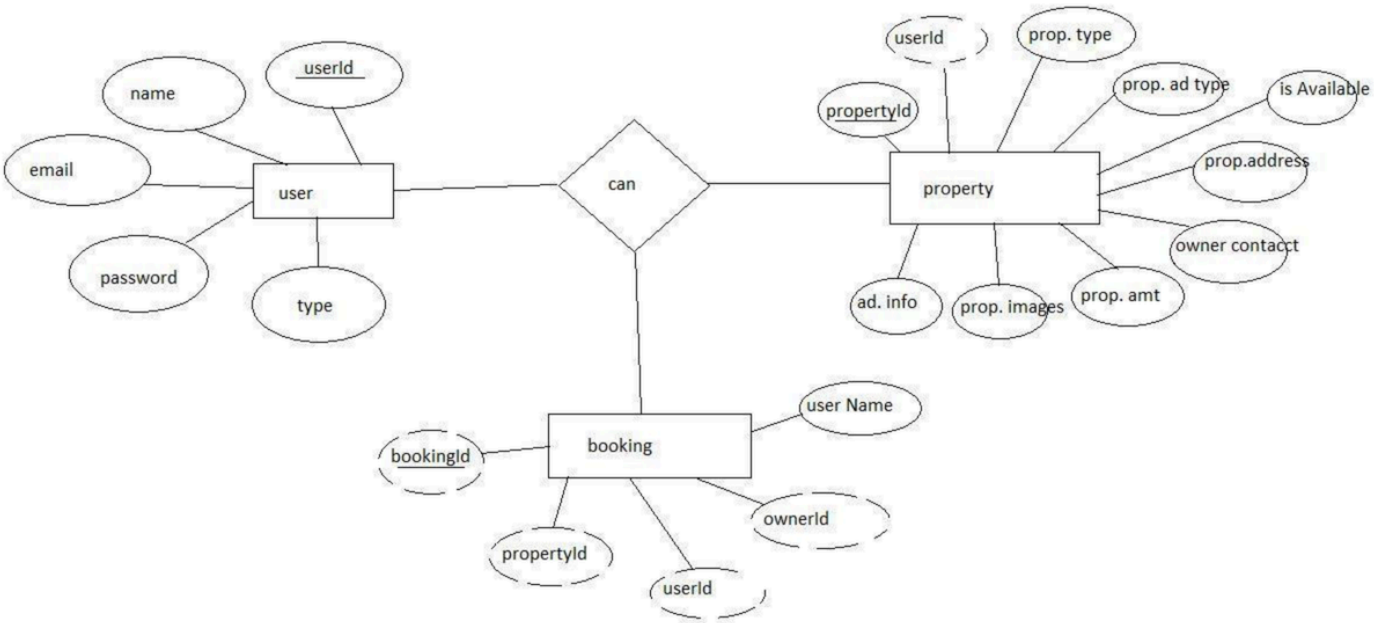
Overall System Integration

The integration of **React (frontend), and Node.js (backend), and MongoDB (database)** forms a complete **MERN stack architecture**. This architecture allows the application to support:

- Real-time communication between client and server
- Efficient data exchange using RESTful APIs
- Scalable and flexible data management
- Responsive and interactive user interfaces

Together, these technologies create a robust and scalable system that ensures a **smooth and efficient house rental experience**, enabling users to browse listings, add properties, and book rental rooms easily.

Explanation of ER Diagram



- **Entities & Attributes**

1. **User**

Attributes:

UserID (PK), Name, Email, Phone, Password, UserType (Tenant / Landlord / Agent), ProfileImage, CurrentLocation.

Explanation:

Represents all users of the app. UserType helps distinguish between renters and property owners.

- **Property**

2. **Attributes:**

PropertyID (PK), OwnerID (FK → User), Title, Description, Location, RentAmount, PropertyType, FurnishingStatus, Amenities, Images, Status (Available/Booked)

Explanation:

Contains all details of a listed property. Linked to the User who owns it.

3. **Booking**

Attributes:

BookingID (PK), PropertyID (FK), TenantID (FK → User), StartDate, EndDate, BookingDate, Status (Confirmed/Pending/Cancelled)

Explanation:
Represents the booking made by a tenant for a property.

4. **BookingHistory**

Attributes:
HistoryID (PK), UserID (FK), PropertyID (FK), BookingID (FK), ViewedOn

Explanation:
Keeps a record of all previous bookings and views for users

Relationships Overview

User ↔ Property:
One user (landlord) can own many properties (1:N).

User ↔ Booking:
A user (tenant) can make multiple bookings (1:N).

Booking ↔ Property:
One booking is made for one property (N:1).

User ↔ SupportTicket:
Users can raise multiple tickets (1:N).

Features of the House Rent Application

1. Advanced Property Search

The system provides an advanced search functionality that helps users quickly find suitable rental properties based on their preferences.

Property Filters:

Users can filter available properties based on multiple criteria such as location, price range, property type, size, number of bedrooms, and specific amenities like pet-friendly homes, parking space, swimming pool, or garage. This allows users to narrow down the search results and easily find properties that meet their requirements.

Saved Searches:

The platform allows users to save their search preferences for future use. This feature helps users quickly access previously used search criteria and check for new property listings that match their selected filters.

2. Market Insights and Trends

The system also provides useful information that helps users understand the real estate market before making rental or investment decisions.

Price Trends:

The application displays information about property price trends in specific areas. This helps users understand whether prices are increasing or decreasing and assists them in making informed decisions.

Neighbourhood Insights:

Users can view important details about the neighbourhood where the property is located. This includes information about nearby amenities such as schools, hospitals, shopping centers, transportation options, and overall living conditions in the area.

Investment Potential:

The system provides insights related to property value appreciation and potential investment opportunities. This feature is useful for users who are interested in real estate investment and want to evaluate the long-term benefits of a property.

User Roles and Responsibilities

The system consists of **three main types of users**: **Renter (Tenant), Owner, and Admin**. Each user has specific roles and responsibilities within the application to ensure smooth property listing, booking, and management.

1. Renter / Tenant

A **Renter (Tenant)** is a user who searches for and books rental properties through the platform.

Responsibilities and Functionalities:

- Create an account and log in to the system using their email and password.
- View all available rental properties displayed on their dashboard after logging in.
- Search and browse properties based on their preferences.
- Click on the **"Get Info"** option to view complete details about the selected property and the owner.
- Fill out and submit a request form with their personal details to express interest in booking the property.
- View submitted booking requests in the **Booking Section** of their dashboard.
- Track the status of their booking request, which initially appears as **"Pending"** until the owner reviews and updates it.

2. Owner

An **Owner** is a registered user who lists properties for rent on the platform.

Responsibilities and Functionalities:

- Register on the platform and request approval from the **Admin** to obtain Owner privileges.
- After receiving approval, access the owner dashboard to manage property listings.
- Perform **CRUD (Create, Read, Update, Delete)** operations on properties.
- Add new properties with details such as location, price, amenities, and property description.
- Update or edit property details whenever necessary.
- Delete properties that are no longer available for rent.
- Manage booking requests received from renters.
- Update the **status of booking requests** (e.g., approve, reject, or pending).
- Change the **availability status** of their properties when they are booked or unavailable.

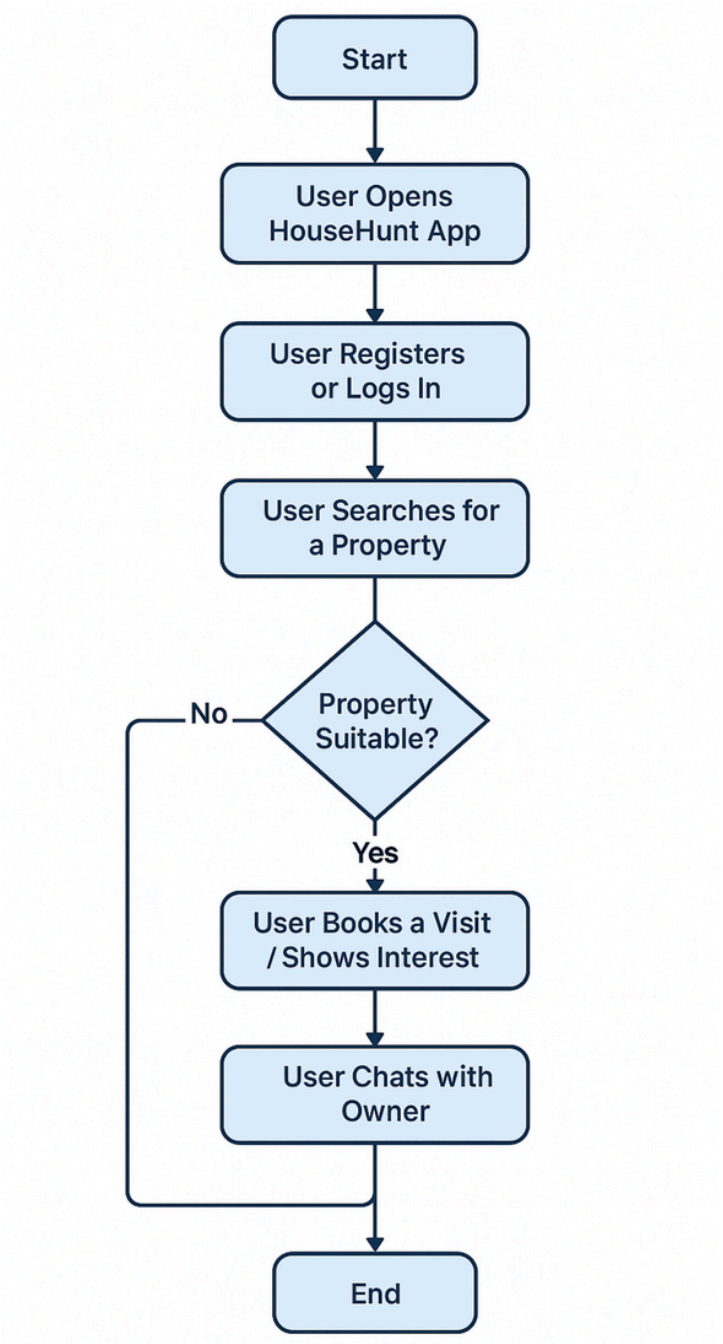
3. Admin

The **Admin** manages the overall functioning of the system and ensures that all platform activities follow the defined rules and policies.

Responsibilities and Functionalities:

- Monitor and manage all users registered on the platform.
- Review and approve legitimate users who request **Owner privileges** so they can list properties on the system.
- Oversee and regulate property listings to ensure the information provided is valid.
- Manage user accounts and maintain platform security.
- Enforce platform policies, terms of service, and privacy regulations.
- Ensure smooth operation of the system by monitoring both renters and owners.

User Flow



This updated user journey reflects only the features that are **actually implemented** in your current HouseHunt application.

1. Start

The user begins their journey to find a rental property.

2. User Opens HouseHunt App

The user launches the HouseHunt app on their smartphone or web browser.

3. User Registers or Logs In

New users can register with basic information.

Existing users can log in using saved credentials.

4. User Searches for a Property

Using search filters like location, rent range, property type, and amenities, the user browses available listings.

5. Property Suitable

If the user finds a suitable property, they proceed to contact the owner.

If not, they continue exploring other listings.

6. User Chats with Owner

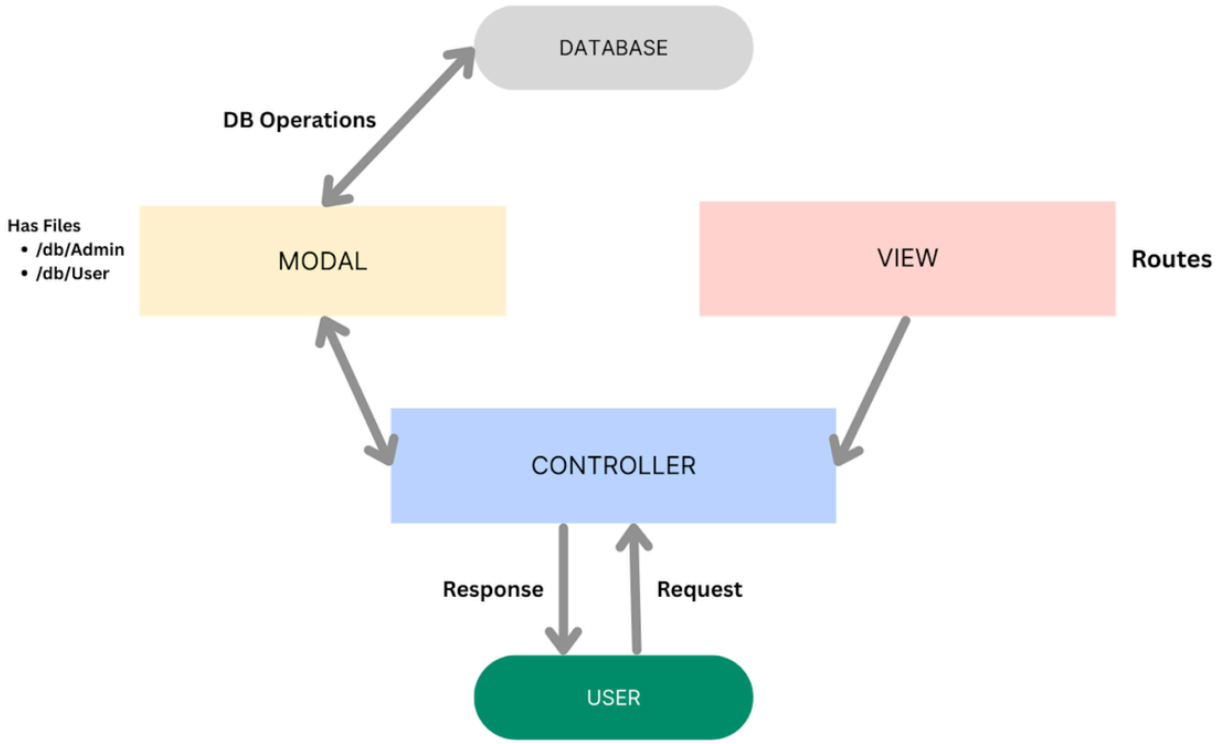
The app enables the user to directly message or contact the property owner for further discussion.

7. End

Once the user finalizes the deal outside the app, their in-app journey ends.

All future communication and formalities (like agreements and payment) happen offline.

MVC PATTERN:



Description

The HouseRent application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.

Scalability: New features can be added easily by creating new routes, controllers, and models.

Reusability: Logic in controllers and models can be reused across multiple parts of the application.

Testing: Each layer can be tested independently, especially the controllers and models.

Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.